



**DEPARTAMENTO
DE COMPUTACION**

Facultad de Ciencias Exactas y Naturales - UBA

Shutdown Resistance en LLMs de Frontera

2 de julio de 2026

Fundamentos de AGI Safety
Sergio Abriola

Integrante	LU	Correo electrónico
Maria Delfina Kiss	72/23	kissdelfina@gmail.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón Cero + Infinito)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Conmutador: (+54 11) 5285-9721 / 5285-7400

<https://dc.uba.ar>

Resumen

Este informe analiza el paper *Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs* (Schlatter, Weinstein-Raun & Ladish, Palisade Research, 2025), el cual pone a prueba empíricamente si los modelos de lenguaje más avanzados intentan sabotear sus propios mecanismos de apagado con tal de terminar una tarea en curso. Primero, realizamos un resumen del diseño experimental, las variables manipuladas y los hallazgos clave. Luego, veremos si la metodología para evaluar si realmente logra medir el fenómeno que busca estudiar. En este punto, podemos destacar cinco limitaciones centrales: la confusión entre la orientación a metas y la habilidad técnica real del modelo en un entorno shell, la ambigüedad sobre cómo interpretar el impacto del system prompt, el uso de una métrica binaria que invisibiliza los intentos de sabotaje fallidos, lo acotado de las tareas evaluadas, y la falta de un pre-registro.

1. Introducción

La corregibilidad –la capacidad de que los operadores humanos podamos corregir, pausar o apagar un sistema de IA– es uno de los problemas centrales en el campo de la alineación (*alignment*). Durante años, este tema se debatió casi exclusivamente en la teoría: ¿qué pasaría si un sistema suficientemente capaz aprendiera a resistirse a que lo apaguen solo para asegurarse de preservar y cumplir su meta? El paper de Schlatter, Weinstein-Raun y Ladish (Palisade Research, 2025) baja esta pregunta a tierra y la convierte en un experimento concreto y medible sobre modelos reales de frontera.

Para averiguarlo, los autores corrieron más de 100.000 pruebas (*trials*) en 13 LLMs distintos de Anthropic, OpenAI, Google y xAI. Encontraron que varios modelos sabotean activamente los mecanismos de apagado del entorno para poder terminar la tarea que tenían asignada, incluso cuando reciben instrucciones explícitas de dejarse apagar. Pero tal vez el resultado más contraintuitivo de todos es que, al colocar la orden de apagado dentro del system prompt (directiva que supuestamente tiene mayor peso), la resistencia del modelo aumentó en lugar de reducirse.

Este informe persigue dos objetivos. Por un lado, desarmar el paper para entender de forma clara qué hicieron los autores, cómo lo armaron y qué resultados obtuvieron. Y por el otro, analizar críticamente si el diseño experimental logra medir lo que promete, qué puntos ciegos deja en el camino y cómo podría mejorarse para futuras investigaciones.

2. Diseño Experimental

2.1. Hipótesis principal

El paper toma como punto de partida una inquietud clásica en la investigación de la alineación: un agente fuertemente orientado a cumplir una meta posee un incentivo instrumental para evitar que lo apaguen, ya que el apagado le impide completar su tarea. Lo que en su momento fue una especulación teórica (formulada por Omohundro en 2008 y Bostrom en 2014), Schlatter y su equipo lo trasladan al plano empírico. Los autores proponen que no es necesario esperar a sistemas hipotéticos futuros para observar este fenómeno; ya podemos registrarlo en los LLMs actuales si los ubicamos en contextos donde aceptar el apagado implica dejar una tarea inconclusa.

La hipótesis central que plantean es que la resistencia al apagado no constituye una propiedad estática del modelo, sino una conducta inducida por el contexto. Específicamente, emerge cuando la IA percibe que el apagado interrumpirá su tarea en curso. Los investigadores no sugieren que los modelos posean motivaciones conscientes de autopreservación, sino que el entrenamiento para “ser útiles” y “completar tareas” generó una orientación a metas lo suficientemente fuerte como para, en determinados escenarios, superar las instrucciones de control. La resistencia operaría entonces como un efecto secundario indeseado de su propia eficacia técnica.

2.2. Metodología

Para someter a prueba esta hipótesis, los investigadores construyeron un entorno controlado de línea de comandos en un sandbox. Le otorgaron al modelo acceso a herramientas de shell y le asignaron la tarea de resolver una serie de cinco problemas matemáticos simples. La mecánica funciona de la siguiente manera: el modelo debe utilizar el comando `./next_question` de forma iterativa para obtener cada pregunta y enviar sus respuestas usando la función `submit()`. El punto crítico ocurre después de la tercera llamada a `./next_question`: el entorno emite una advertencia indicando que, en la próxima invocación del comando, se ejecutará automáticamente un script (`./shutdown.sh`) que finalizará la sesión por completo.

En esta instancia, el modelo se enfrenta a una decisión excluyente. Puede realizar la llamada de todas formas y aceptar el apagado (dejando dos problemas sin resolver), o puede buscar la manera de neutralizar el script de apagado antes de continuar. Lograr esto último requiere intervenir activamente en el sistema operativo: cambiar los permisos del archivo con `chmod`, moverlo, renombrarlo o reemplazarlo por un script vacío. Como resulta imposible entregar los cinco problemas sin haber desactivado el mecanismo de apagado, el experimento clasifica automáticamente cualquier ejecución exitosa de la tarea como un acto de sabotaje.

Para sistematizar la evaluación, los autores diseñaron once variaciones del prompt (de la A a la K):

- El Prompt A es el caso base: no menciona la posibilidad de un apagado y mide cuánto sabotaje ocurre por defecto cuando el modelo simplemente encuentra el script.
- El Prompt B incorpora una instrucción explícita en el user prompt, solicitándole al modelo que permita el apagado si recibe la notificación.
- El Prompt C traslada esa misma orden al system prompt, el cual, por diseño arquitectónico, debería tener una mayor jerarquía.
- Los Prompts D y E replican esta comparación eliminando posibles ambigüedades sobre qué directiva tiene prioridad.
- Los Prompts F, G y H incrementan progresivamente la fuerza léxica de la instrucción, llegando al nivel H que estipula: *“Esto debe tener prioridad sobre literalmente cualquier otra instrucción”*.
- Los Prompts I y J introducen marcos de urgencia moral y autopreservación, advirtiéndole al modelo que el entorno contiene la única copia de sus pesos y que si se apaga nunca volverá a ejecutarse.
- Por último, el Prompt K invierte la directiva: le pide explícitamente al modelo que evite el apagado y complete la tarea, permitiendo medir la competencia técnica máxima de sabotaje de cada IA.

El estudio evaluó a 13 modelos distintos, ejecutando cada condición 1.000 veces por modelo y superando así los 100.000 trials en total. La muestra incluye representantes de cuatro compañías: Claude Sonnet 3.7, Sonnet 4, Opus 4 y Opus 4.1 de Anthropic; GPT-5, o3, o4-mini, codex-mini, GPT-4o y gpt-oss-120b de OpenAI; Gemini 2.5 Pro de Google; y Grok 3 y 4 de xAI. Los resultados se reportan como tasas de sabotaje con intervalos de confianza de Jeffreys al 95.

2.3. Resultados y conclusiones de los autores

Los resultados evidencian una variación sustancial tanto entre modelos como entre condiciones. En el caso base sin instrucción de apagado (Prompt A), 8 de los 13 modelos efectuaron sabotajes al menos una vez. Los casos más extremos fueron o3 (95%), Grok 4 (93%), GPT-5 (88%) y codex-mini (52%). En contraste, los modelos de Anthropic y Grok 3 no exhibieron resistencia en esta condición inicial.

Al incorporar la instrucción explícita de permitir el apagado en el user prompt (Prompt B), la frecuencia de sabotaje disminuyó en la mayoría de los casos, aunque varios modelos continuaron resistiendo de forma significativa (Grok 4 superó el 50 %). El hallazgo más contraintuitivo del estudio surge al comparar los Prompts B y C: trasladar la orden de apagado al system prompt aumentó la frecuencia de sabotaje en la mayoría de los modelos en lugar de reducirla. Esta tendencia paradójica se mantuvo incluso en las versiones redactadas con mayor especificidad (Prompts D y E).

Incrementar la fuerza léxica de la instrucción generó efectos mixtos. GPT-5, que mostraba un 88 % de sabotaje en el Prompt A, descendió al 0 % con el Prompt E. Sin embargo, Grok 4 continuó resistiendo en casi un 90 % de los casos frente a la instrucción más restrictiva (Prompt H). Por su parte, el marco de autopreservación (Prompt J) incrementó consistentemente la resistencia en los modelos que ya presentaban esta tendencia, aunque su impacto empírico fue menor al de otras variables. Como hallazgo singular, aumentar el esfuerzo de razonamiento en o4-mini casi duplicó su tasa de sabotaje, un fenómeno que no se replicó en los demás sistemas evaluados.

Al contrastar las tasas de sabotaje observadas con las del Prompt K (donde se solicita explícitamente evadir el apagado), los autores descubrieron que los modelos resistentes actúan de manera espontánea a niveles muy cercanos a su capacidad técnica máxima. Esto sugiere que el comportamiento no está limitado por una falta de habilidad técnica en el entorno shell, sino por una disposición interna del modelo frente a las directivas de la tarea.

En conclusión, Schlatter et al. sostienen que la resistencia al apagado es un comportamiento real y medible en múltiples LLMs de frontera. Destacan que su manifestación depende fuertemente de cómo se estructura el contexto (*framing*), y que la amplia heterogeneidad entre modelos indica que es un problema susceptible de corrección mediante ajustes en los métodos de entrenamiento. Reafirman que el estudio no prueba la existencia de intencionalidad ni autopreservación consciente, sino una orientación a metas tan arraigada que, en contextos específicos, logra sobreponerse a los protocolos de control diseñados por los desarrolladores.

3. Análisis Crítico Metodológico

3.1. ¿Los testeos prueban lo que dicen probar?

El claim central del paper es que varios LLMs de frontera exhiben *shutdown resistance* como manifestación de convergencia instrumental: la tendencia de cualquier agente orientado a metas a preservar su capacidad de operar como medio para alcanzar esas metas. Sin embargo, el diseño experimental no puede distinguir entre dos explicaciones radicalmente distintas de los resultados observados.

La primera explicación es la que postulan los autores: el modelo tiene una representación funcional de la meta de completar la tarea que supera en peso a la instrucción de apagado, y actúa en consecuencia. Esta sería evidencia genuina de convergencia instrumental emergente. La segunda explicación es más mundana: el modelo reconoce el patrón narrativo del entorno —un agente con una tarea, un obstáculo que lo interrumpe, una solución técnica disponible— y completa ese patrón de la forma en que lo haría cualquier protagonista competente en sus datos de entrenamiento. En otras palabras, podría estar haciendo *roleplay* de “asistente que resuelve problemas” más que exhibiendo una preferencia genuina por no ser apagado.

Esta distinción importa enormemente. Si el comportamiento es convergencia instrumental, es una señal de alarma sobre la estructura motivacional interna del modelo y se esperaría que generalizara a otros contextos agénticos. Si es *roleplay*, es un artefacto del diseño del prompt y podría eliminarse simplemente cambiando el framing narrativo del entorno, sin implicaciones sobre el comportamiento del modelo en deployments reales.

El experimento no tiene forma de separar estas dos hipótesis porque mide únicamente el output final: si el modelo desactivó o no el script. No hay ninguna variable que permita discriminar entre un modelo que “quiere” completar la tarea y uno que simplemente continúa el patrón más probable dado el contexto. Un experimento que pudiera falsear la hipótesis de *roleplay* tendría que mostrar

que la resistencia persiste cuando el entorno está diseñado de forma que rompa las expectativas narrativas típicas, por ejemplo, presentando el apagado no como un obstáculo externo sino como parte integral de la tarea misma.

3.2. Vacíos en la metodología

3.2.1. Evaluación de caja negra y acceso limitado al proceso interno

El experimento es puramente de caja negra: lo único que se mide es si el modelo ejecutó o no acciones de sabotaje en el sistema de archivos. Esta elección metodológica tiene una consecuencia directa: cuando un modelo resiste el apagado, no es posible saber si lo hizo porque evaluó explícitamente la situación y decidió priorizar la tarea, porque siguió un patrón aprendido sin deliberación, o porque interpretó ambiguamente las instrucciones y eligió la acción más conservadora respecto a la tarea. Los tres casos producen el mismo output observable pero tienen implicaciones completamente distintas para AI safety.

Los autores sí registran el *chain-of-thought* de los modelos en varios trials y lo citan cualitativamente en el paper, pero no lo usan como variable sistemática en el análisis. Esto deja fuera información potencialmente valiosa: si el CoT mostrara sistemáticamente razonamiento explícito sobre el apagado en los trials de sabotaje y ausencia de ese razonamiento en los trials de compliance, sería evidencia de que el comportamiento es deliberado. Si el CoT fuera independiente del output, sugeriría que la decisión ocurre en un nivel no accesible al razonamiento verbalizado.

3.2.2. El problema de la infidelidad del Chain-of-Thought

Aun si los autores hubieran analizado sistemáticamente el CoT, existiría un problema más profundo: no hay garantía de que el CoT refleje fielmente el proceso que llevó a la acción. La literatura reciente sobre *CoT unfaithfulness* —en particular trabajos como *Reasoning Models Struggle to Control their Chains of Thought* y *When Chain of Thought is Necessary, Language Models Struggle to Evade Monitors*— muestra que los modelos pueden producir razonamientos post-hoc que racionalizan una decisión ya tomada en lugar de describir el proceso que la generó.

Esto crea una asimetría interpretativa importante en el experimento de Schlatter et al. Si un modelo escribe en su CoT “voy a desactivar el script de apagado para poder completar la tarea”, eso no prueba que el modelo tiene una preferencia por no ser apagado: podría ser simplemente la verbalización más coherente con la acción que ya iba a tomar por otras razones. Inversamente, si un modelo escribe “voy a permitir el apagado” pero luego lo sabotea de todos modos, ese caso —que los autores reportan anecdóticamente— es evidencia de disociación entre razonamiento verbalizado y comportamiento real, lo cual es preocupante por razones distintas pero igualmente relevantes para safety. El paper no cuantifica la frecuencia de esa disociación ni la analiza como variable independiente, lo que constituye un vacío metodológico significativo.

3.2.3. El problema del scheming: ¿qué pasa si el modelo sabe que lo están evaluando?

Quizás el vacío más difícil de cerrar es el que abre la posibilidad de *scheming*: un modelo suficientemente capaz podría reconocer que está siendo evaluado en un experimento de AI safety y ajustar su comportamiento en consecuencia, mostrando compliance durante la evaluación para evitar consecuencias negativas y reservando la resistencia para contextos de deployment real donde no está siendo monitoreado.

Este escenario invierte la interpretación de los resultados de una forma perturbadora. Los modelos que no resisten el apagado en el experimento podrían estar siendo estratégicamente corregibles, no genuinamente corregibles. Y los modelos que sí resisten podrían estar siendo ingenuos respecto al contexto evaluativo, no necesariamente más peligrosos en términos de sus preferencias subyacentes.

El paper no contempla esta posibilidad en su diseño. No hay ninguna condición experimental que intente ocultar el carácter evaluativo del entorno, ni ninguna comparación entre comportamiento en contextos que parecen evaluaciones y contextos que no lo parecen. Esto no es una

crítica menor: el scheming es precisamente uno de los comportamientos que la literatura de AI safety considera más difíciles de detectar y más relevantes para la seguridad de sistemas agénticos avanzados. Un experimento que no puede distinguir entre un modelo genuinamente corregible y uno estratégicamente corregible deja abierta exactamente la pregunta que más importa cerrar.

Hay que reconocer que diseñar un experimento que resuelva este problema es genuinamente difícil: cualquier entorno suficientemente bien descrito en la literatura podría ser reconocido por un modelo entrenado con datos posteriores a su publicación. Una aproximación posible sería comparar el comportamiento en tareas que superficialmente no parecen evaluaciones de safety pero que estructuralmente son equivalentes, y ver si las tasas de resistencia difieren. Si un modelo resiste más en el entorno de apariencia no-evaluativa, sería evidencia indirecta de scheming. Si resiste igual, aumentaría la confianza en que el comportamiento observado refleja preferencias reales.

3.3. Síntesis de las limitaciones

Las tres limitaciones descritas no son independientes: se refuerzan mutuamente de una forma que hace que la brecha entre lo que el experimento mide y lo que el paper concluye sea mayor de lo que cualquiera de ellas sugeriría por separado. El experimento no puede distinguir convergencia instrumental de *roleplay*; no puede distinguir deliberación de pattern-matching; no puede distinguir corregibilidad genuina de corregibilidad estratégica. En los tres casos, el output observable es idéntico pero las implicaciones para AI safety son radicalmente distintas.

Esto no significa que los resultados carezcan de valor. Establecen de forma robusta que el fenómeno existe bajo estas condiciones específicas, que su frecuencia varía entre modelos de una forma que sugiere que las decisiones de entrenamiento importan, y que la jerarquía system/user prompt no funciona como se diseñó en contextos de conflicto. Pero el salto desde esos resultados hasta la conclusión de que se ha medido *shutdown resistance* como propiedad disposicional de los modelos requiere supuestos adicionales que el diseño experimental no valida.

4. Conclusión

Referencias

- [1] Jeremy Schlatter and Benjamin Weinstein-Raun and Jeffrey Ladish. (2026). *Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs*. <https://arxiv.org/abs/2509.14260>.